

EXPERIMENT 19

IMPLEMENTATION OF HASH FUNCTION – SHA-1

AIM:

To implement the SHA-1 (Secure Hash Algorithm 1) hash function in C and generate the 160-bit message digest for a given input message.

ALGORITHM:

1. Accept the input message from the user.
2. Convert the message into a sequence of bytes.
3. Append a single 1 bit to the message.
4. Append 0 bits until the message length is 448 bits modulo 512.
5. Append the original message length as a 64-bit value.
6. Divide the padded message into 512-bit blocks.
7. Initialize five 32-bit hash values: H0, H1, H2, H3, and H4.
8. Expand each 512-bit block into 80 words.
9. Perform 80 rounds of processing using four logical functions.
10. Add the resulting values to the current hash values.
11. Process all message blocks.
12. Concatenate H0, H1, H2, H3, and H4 to produce the 160-bit SHA-1 hash.

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
uint32_t leftRotate(uint32_t x, uint32_t n)
```

```
{
```

```
    return (x << n) | (x >> (32 - n));
```

```
}
```

```

void sha1(const unsigned char *message, size_t length)
{
    uint32_t H0 = 0x67452301;

    uint32_t H1 = 0xEFCDAB89;

    uint32_t H2 = 0x98BADCFE;

    uint32_t H3 = 0x10325476;

    uint32_t H4 = 0xC3D2E1F0;


    uint64_t bitLength = (uint64_t)length * 8;


    /* Calculate padded length */

    size_t newLength = length + 1;


    while (newLength % 64 != 56)

        newLength++;


    size_t totalLength = newLength + 8;


    unsigned char *data =

        (unsigned char *)calloc(totalLength, 1);


    if (data == NULL)

    {

```

```

    printf("Memory allocation failed.\n");

    return;
}

/* Copy message */

memcpy(data, message, length);

/* Append 1 bit */

data[length] = 0x80;

/* Append original length in bits */

for (int i = 0; i < 8; i++)
{
    data[newLength + i] =
        (unsigned char)(bitLength >> (56 - 8 * i));
}

/* Process each 512-bit block */

for (size_t offset = 0;
     offset < totalLength;
     offset += 64)
{
    uint32_t W[80];

```

```
/* First 16 words */
```

```
for (int i = 0; i < 16; i++)
```

```
{
```

```
    W[i] =
```

```
        ((uint32_t)data[offset + i * 4] << 24) |
```

```
        ((uint32_t)data[offset + i * 4 + 1] << 16) |
```

```
        ((uint32_t)data[offset + i * 4 + 2] << 8) |
```

```
        ((uint32_t)data[offset + i * 4 + 3]);
```

```
}
```

```
/* Expand to 80 words */
```

```
for (int i = 16; i < 80; i++)
```

```
{
```

```
    W[i] = leftRotate(
```

```
        W[i - 3] ^
```

```
        W[i - 8] ^
```

```
        W[i - 14] ^
```

```
        W[i - 16],
```

```
        1
```

```
    );
```

```
}
```

```
uint32_t A = H0;
```

```
uint32_t B = H1;
```

```
uint32_t C = H2;
```

```
uint32_t D = H3;
```

```
uint32_t E = H4;
```

```
/* 80 rounds */
```

```
for (int i = 0; i < 80; i++)
```

```
{
```

```
    uint32_t F;
```

```
    uint32_t K;
```

```
    if (i < 20)
```

```
    {
```

```
        F = (B & C) | ((~B) & D);
```

```
        K = 0x5A827999;
```

```
    }
```

```
    else if (i < 40)
```

```
    {
```

```
        F = B ^ C ^ D;
```

```
        K = 0x6ED9EBA1;
```

```
    }
```

```
    else if (i < 60)
```

```
{  
  
    F = (B & C) |  
        (B & D) |  
        (C & D);  
  
    K = 0x8F1BBCDC;
```

```
}
```

```
else
```

```
{  
  
    F = B ^ C ^ D;  
  
    K = 0xCA62C1D6;  
  
}
```

```
uint32_t TEMP =
```

```
    leftRotate(A, 5)
```

```
    + F
```

```
    + E
```

```
    + K
```

```
    + W[i];
```

```
E = D;
```

```
D = C;
```

```
C = leftRotate(B, 30);
```

```
B = A;
```

```
        A = TEMP;

    }

    H0 += A;

    H1 += B;

    H2 += C;

    H3 += D;

    H4 += E;

}

free(data);

printf("\nSHA-1 HASH\n");

printf("-----\n");

printf("%08x%08x%08x%08x%08x\n",

        H0, H1, H2, H3, H4);

}

int main()

{

    char message[1000];
```

```

printf("SHA-1 HASH FUNCTION\n");

printf("-----\n");

printf("Enter message: ");

fgets(message, sizeof(message), stdin);

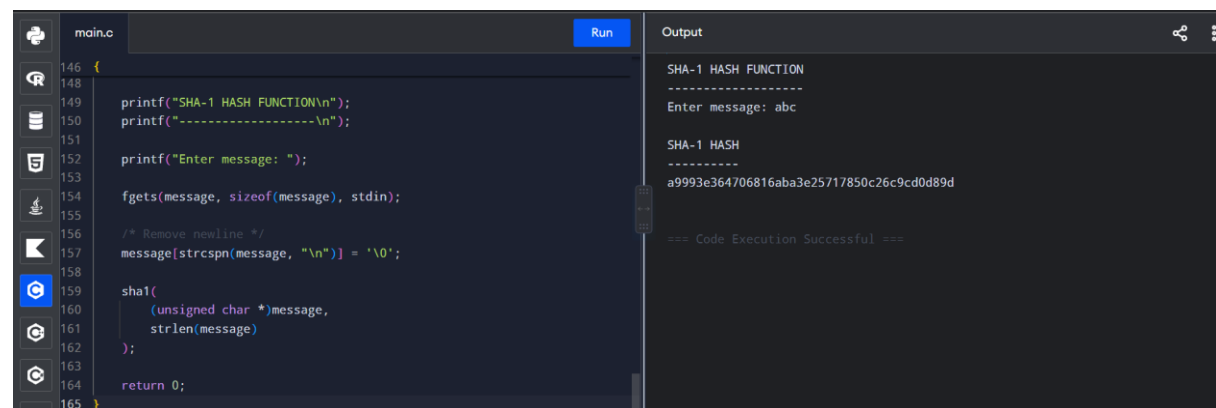
/* Remove newline */
message[strcspn(message, "\n")] = '\0';

sha1(
    (unsigned char *)message,
    strlen(message)
);

return 0;
}

```

OUTPUT:



The screenshot shows a code editor with a file named `main.c` and a `Run` button. The code in the editor is as follows:

```

146 {
148
149     printf("SHA-1 HASH FUNCTION\n");
150     printf("-----\n");
151
152     printf("Enter message: ");
153
154     fgets(message, sizeof(message), stdin);
155
156     /* Remove newline */
157     message[strcspn(message, "\n")] = '\0';
158
159     sha1(
160         (unsigned char *)message,
161         strlen(message)
162     );
163
164     return 0;
165 }

```

The output window on the right shows the following text:

```

SHA-1 HASH FUNCTION
-----
Enter message: abc

SHA-1 HASH
-----
a9993e364706816aba3e25717850c26c9cd0d89d

=== Code Execution Successful ===

```

RESULT:

Thus, the SHA-1 hash function was successfully implemented in C, and a 160-bit message digest was generated for the given input message.